



TALLER 4

SISTEMA DE BÚSQUEDA Y RECUPERACIÓN DE ARTÍCULOS CIENTÍFICOS

Comparación de TF-IDF y LSA integrada en una aplicación Shiny

Link del despliegue: <https://4jqmox-luisa0fernanda-guerrero0ordonez.shinyapps.io/bdcc-research-dashboard-taller4/>

Estudiante: Luisa Fernanda Guerrero Ordoñez

Asignatura: Minería de Datos

Fecha de entrega: 30 de julio de 2026

Ciudad: Bogotá D. C.

Resumen

El objetivo de este trabajo fue ampliar la aplicación desarrollada en el Taller 2 mediante un sistema de búsqueda y recuperación de artículos científicos. La aplicación recibe una consulta escrita en lenguaje natural y devuelve una lista de artículos ordenados según su similitud con el tema consultado. Para construir el buscador se utilizó la información textual almacenada en una base SQLite, principalmente el título y el resumen de cada artículo. Después de preparar los textos, se implementaron dos estrategias: TF-IDF con similitud coseno y Latent Semantic Analysis (LSA) con similitud coseno.

TF-IDF se utilizó como estrategia léxica porque permite dar mayor peso a los términos que ayudan a diferenciar un artículo. LSA se utilizó como segunda estrategia porque reduce la representación original a 200 componentes y permite comparar los documentos dentro de un espacio más compacto. La consulta del usuario se procesa con las mismas reglas aplicadas al corpus y, en el caso de LSA, se proyecta al mismo espacio reducido antes de calcular la similitud.

La evaluación se realizó con cinco consultas y una revisión manual de los cinco primeros resultados de cada estrategia. Ambas obtuvieron una Precisión promedio de 0,92 y un MRR de 0,90. Aunque las métricas globales fueron iguales, los rankings no fueron idénticos, lo que permitió observar diferencias entre una recuperación basada en términos y una recuperación basada en componentes latentes. Finalmente, los modelos se integraron en Shiny mediante objetos previamente guardados, evitando reconstruir la matriz y entrenar la reducción en cada búsqueda.

Palabras clave: recuperación de información, TF-IDF, LSA, similitud coseno, reducción dimensional, Shiny.

Contenido

- 1. Introducción
- 2. Planteamiento del problema
- 3. Objetivos
- 4. Descripción de la base de datos
- 5. Construcción del corpus
- 6. Procesamiento del texto
- 7. Representación vectorial con TF-IDF
- 8. Similitud coseno y mecanismo de ranking
- 9. Reducción dimensional con LSA
- 10. Selección de 200 componentes
- 11. Estrategia de recuperación TF-IDF
- 12. Estrategia de recuperación LSA
- 13. Comparación de estrategias
- 14. Evaluación de resultados
- 15. Análisis de resultados incorrectos
- 16. Integración y funcionamiento de la aplicación
- 17. Consideraciones computacionales
- 18. Reproducibilidad y estructura del proyecto
- 19. Limitaciones
- 20. Conclusiones
- 21. Referencias
- Anexos

1. Introducción

La búsqueda de información científica no consiste solamente en localizar una palabra dentro de una base. Una persona puede conocer el tema que desea estudiar, pero no el título exacto de los artículos ni la forma en que sus autores expresaron esa idea. Por ejemplo, una consulta como “machine learning for disease diagnosis” puede estar relacionada con documentos que usan expresiones como prediction, detection, clinical decision support o medical classification.

En el Taller 2 se había construido una aplicación Shiny para consultar y visualizar artículos de la revista Big Data and Cognitive Computing. Esa versión permitía explorar la base mediante filtros, indicadores, gráficos y una tabla dinámica. El Taller 4 conserva esas funciones y agrega un buscador capaz de recibir consultas en lenguaje natural.

Para realizar el ranking fue necesario convertir el texto en números. Se compararon dos alternativas. La primera fue TF-IDF, que representa la importancia de cada palabra dentro de un artículo y dentro del corpus. La segunda fue LSA, que parte de TF-IDF y reduce la matriz mediante Truncated SVD. Las dos estrategias utilizan similitud coseno para ordenar los artículos.

Propósito del informe

Documentar de forma completa cómo se pasó desde los textos almacenados en SQLite hasta los rankings que utiliza la aplicación, justificando las decisiones tomadas y explicando sus ventajas, limitaciones y resultados.

2. Planteamiento del problema

Los filtros tradicionales son adecuados cuando el usuario conoce un autor, una categoría o un año. Sin embargo, no resuelven por sí solos una consulta temática escrita como una frase. Una búsqueda con SQL LIKE o coincidencia exacta puede encontrar documentos que contienen una palabra, pero no establece una medida clara para ordenarlos según relevancia.

El problema se formuló como una tarea de recuperación de información: dado un conjunto de artículos y una consulta, se debe asignar un puntaje a cada documento, ordenar los puntajes de mayor a menor y mostrar los primeros resultados. El ranking principal no puede depender únicamente de la fecha, las citas o las descargas; debe provenir de una representación numérica del contenido textual.

También se necesitaba comparar una estrategia sin reducción y otra con reducción dimensional. La reducción no debía quedar como un ejercicio aislado, sino formar parte de uno de los rankings reales de la aplicación.

3. Objetivos

3.1 Objetivo general

Desarrollar e integrar en una aplicación Shiny un sistema de recuperación de artículos científicos que permita realizar consultas en lenguaje natural y obtener resultados ordenados por relevancia, comparando TF-IDF y LSA.

3.2 Objetivos específicos

- Examinar la información textual disponible en la base SQLite.
- Construir un corpus que represente cada artículo mediante sus campos textuales.
- Preparar los textos y las consultas con las mismas reglas.
- Construir una matriz TF-IDF en formato disperso.
- Aplicar Truncated SVD para obtener una representación LSA de 200 componentes.
- Calcular similitud coseno y generar rankings para las dos estrategias.
- Evaluar los resultados con cinco consultas, Precisión y MRR.
- Analizar resultados relevantes y poco relevantes.
- Integrar los objetos en Shiny sin volver a entrenar los modelos en cada búsqueda.
- Mantener las funcionalidades desarrolladas en el Taller 2.

4. Descripción de la base de datos

La fuente central del proyecto es una base SQLite con artículos de la revista Big Data and Cognitive Computing. SQLite fue apropiado porque almacena la información en un archivo único, no requiere instalar un servidor de base de datos y puede ser consultado directamente desde R mediante DBI y RSQLite.

En la versión utilizada para construir los modelos, el corpus quedó conformado por 493 registros identificados mediante paper_id. La base contiene información bibliográfica y textual. No todos los campos se usan para calcular similitud; algunos se conservan para presentar y explicar los resultados.

Campo	Descripción	Uso
paper_id	Identificador interno	Relacionar el ranking con el registro original
title	Título del artículo	Parte principal del corpus y salida del buscador
abstract	Resumen	Aporta contexto y se muestra como fragmento
authors	Autores	Se presenta en Ver detalle
publication_date	Fecha de publicación	Se muestra; criterio secundario en empates
topic_label	Tema o categoría	Ayuda a interpretar el resultado
doi / url	Enlace persistente	Permite abrir el artículo
citations	Cantidad de citas	Información complementaria y desempate

4.1 Revisión de faltantes

Antes de construir el corpus se revisaron títulos, resúmenes y palabras clave. Los faltantes no se reemplazaron con texto inventado. Cuando faltaba un resumen, el artículo podía conservarse si tenía un título válido. Cuando faltaba el título y tampoco había otro texto útil, el registro no debía aportar al corpus.

4.2 Criterios de inclusión y exclusión

- Se incluyeron artículos con al menos un campo textual útil.
- Se conservaron los identificadores y campos bibliográficos para poder mostrar el resultado.
- No se inventaron resúmenes ni palabras clave.
- Se descartaron textos vacíos después de la limpieza.

- Los artículos se representaron con la misma estructura para evitar comparaciones inconsistentes.



Justificación de SQLite

SQLite permite que la aplicación, los scripts y el informe trabajen con la misma fuente. Además, facilita la reproducción porque la base puede incluirse en la carpeta del proyecto.

5. Construcción del corpus

El corpus es la colección de textos que representa los artículos. Para cada registro se combinaron principalmente el título y el resumen. Se tomó esta decisión porque el título suele indicar el tema central, pero puede ser demasiado corto; el resumen contiene más contexto, aunque también incluye palabras generales. La combinación permite aprovechar las dos fuentes.

```
corpus <- articulos %>%
  mutate(
    title_limpio = coalesce(title, ""),
    abstract_limpio = coalesce(abstract, ""),
    texto_documento = str_squish(
      paste(title_limpio, abstract_limpio)
    )
  ) %>%
  filter(texto_documento != "")
```

5.1 Decisiones tomadas

Decisión	Justificación
Título + resumen	Aporta una descripción corta y otra más amplia del contenido.
Autores fuera del corpus	Los nombres no describen el tema y podrían introducir coincidencias poco útiles.
Fecha fuera del puntaje principal	Una fecha reciente no significa que el artículo sea más relevante para la consulta.
Citas fuera del puntaje principal	La popularidad no reemplaza la similitud temática.
DOI fuera del corpus	Es un identificador y no una descripción del contenido.
Palabras clave según disponibilidad	Solo se incluyen cuando existen; no se inventan para completar registros.

6. Procesamiento del texto

El procesamiento busca reducir diferencias de escritura que no representan diferencias de significado. Se evitó aplicar una limpieza excesiva, porque algunos términos científicos, siglas y expresiones con guion pueden ser importantes.

Técnica	Justificación
Conversión a minúsculas	Evitar que Machine y machine se conviertan en columnas distintas.
Eliminación de etiquetas HTML	Retirar marcas de formato presentes en algunos resúmenes.
Normalización de entidades	Convertir o retirar códigos como & o .
Puntuación	Reemplazar signos por espacios para separar términos.
Números aislados	Eliminar cifras sin contexto; se conservan términos alfanuméricos cuando forman una unidad útil.
Espacios	Unificar saltos de línea, tabulaciones y espacios repetidos.
Tokenización	Dividir el texto en unidades que puedan contarse.
Stopwords	Reducir palabras muy frecuentes con bajo contenido temático.
Unigramas	Controlar el tamaño del vocabulario y mantener una representación fácil de interpretar.

6.1 Decisiones que no se aplicaron

No se aplicó stemming agresivo porque puede deformar términos técnicos y dificultar la interpretación. Tampoco se utilizaron modelos de lenguaje grandes, ya que no eran necesarios para cumplir el objetivo y aumentaban las dependencias, el tiempo y la memoria. Los bigramas no se utilizaron como representación principal para mantener controlado el vocabulario.

```
normalizar_consulta <- function(texto) {
  texto %>%
    str_replace_all("<[^>]+>", " ") %>%
    str_to_lower() %>%
    str_replace_all("[^[:alnum:]]\\-", " ") %>%
    str_replace_all(
      "(?

```

Regla de consistencia

La consulta del usuario se procesa con la misma función utilizada para los documentos. Si se limpiara de una forma distinta, los vectores dejarían de ser directamente comparables.

7. Representación vectorial con TF-IDF

Después de tokenizar los documentos se construyó una matriz documento-término. Cada fila representa un artículo y cada columna un término del vocabulario. Sobre esta matriz se aplicó TF-IDF.

TF mide la frecuencia de un término dentro de un artículo. IDF disminuye el peso de palabras que aparecen en muchos documentos. El producto permite destacar términos característicos. Por ejemplo, machine puede aparecer en muchos artículos, mientras que intrusion o Parkinson pueden ser más discriminantes en una consulta específica.



$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

7.1 Formato disperso

La matriz TF-IDF se mantuvo en formato disperso porque cada artículo utiliza solo una parte del vocabulario. Convertir toda la matriz a formato denso reservaría memoria para una gran cantidad de ceros. La decisión de conservar el formato disperso permite trabajar dentro del límite de memoria de un computador personal.

7.2 Dimensión original

La representación original tiene 493 filas y una cantidad de columnas igual al vocabulario final. El tamaño exacto del vocabulario se obtiene del objeto `vocabulario_tfidf.rds` y se reporta en el documento reproducible mediante `ncol(matriz_tfidf)`. Se evita escribir manualmente esa cifra en el código para que el informe siga siendo correcto si la base cambia.

7.3 Ventajas y limitaciones

Ventajas de TF-IDF	Limitaciones de TF-IDF
Es sencillo de explicar y revisar.	Depende bastante de las palabras utilizadas.
Funciona bien con términos directos.	No reconoce sinónimos de forma explícita.
La matriz dispersa reduce memoria.	No comprende el significado como una persona.
El ranking se calcula rápidamente.	Puede recuperar textos que repiten términos sin responder exactamente a la consulta.

8. Similitud coseno y mecanismo de ranking

La similitud coseno compara la orientación del vector de la consulta con el vector de cada artículo. Dos vectores con una dirección parecida reciben un puntaje mayor. La medida es útil porque evita que los documentos largos sean favorecidos únicamente por tener más palabras.

$$\text{sim}(q,d) = (q \cdot d) / (\|q\| \|d\|)$$

8.1 Cómo se construye el ranking

1. Se recibe la consulta en lenguaje natural.
2. Se normaliza y tokeniza con las mismas reglas del corpus.
3. Se transforma usando el vocabulario y el modelo TF-IDF ya construidos.
4. Se calcula la similitud con cada artículo.
5. Se ordenan los puntajes de mayor a menor.
6. Se muestran los primeros 5, 10 o 20 resultados.

8.2 Manejo de empates

Cuando dos artículos obtienen el mismo puntaje, se utilizan criterios secundarios: citas, fecha y título. Estos campos no reemplazan la similitud; solamente permiten obtener un orden estable cuando el puntaje principal es igual.

Interpretación del puntaje

El puntaje de similitud no es una probabilidad. Un valor mayor significa que el artículo es más cercano a la consulta dentro de la representación utilizada.

9. Reducción dimensional con LSA

La matriz TF-IDF puede contener muchas columnas. Esta alta dimensionalidad aumenta el tamaño del objeto y mantiene relaciones separadas entre términos que podrían aparecer en contextos parecidos. Para construir la segunda estrategia se aplicó Latent Semantic Analysis mediante Truncated SVD.

Truncated SVD descompone la matriz y conserva un número limitado de componentes. Cada componente resume patrones de coaparición entre términos y documentos. El resultado ya no se interpreta como una columna por palabra, sino como una posición dentro de un espacio latente.

9.1 Por qué se utilizó Truncated SVD

- Puede aplicarse a matrices dispersas de texto.
- No exige convertir toda la matriz TF-IDF a formato denso antes de reducirla.
- Es una técnica estándar para construir LSA.
- Permite guardar una representación más compacta de los documentos.
- La misma transformación puede aplicarse a una consulta nueva.

9.2 Qué información intenta conservar

La reducción intenta conservar los patrones principales de asociación entre documentos y términos. Por ejemplo, si diagnosis, prediction y clinical aparecen en contextos relacionados, una componente puede representar parte de esa relación. No se conserva cada peso original de manera exacta.

9.3 Qué información puede perderse

Al reducir miles de columnas a 200 componentes se pierde detalle. Un término muy específico puede quedar diluido si aparece pocas veces. También puede ocurrir que artículos relacionados de forma general suban en el ranking, aunque no respondan exactamente a la consulta. Por eso la reducción se evaluó dentro del buscador y no se asumió que era mejor por sí sola.

10. Selección de 200 componentes

La dimensión final se estableció en 200 componentes. Esta elección se justificó como un equilibrio entre cuatro aspectos: conservación de información, memoria, tiempo de respuesta y estabilidad del ranking.

Criterio	Interpretación
Conservación de información	Una cantidad muy baja podía eliminar relaciones útiles.

Memoria	200 columnas representan una reducción importante frente al vocabulario completo.
Tiempo	La comparación en un espacio de 200 componentes sigue siendo rápida.
Calidad del ranking	Se verificó que los resultados continuaran siendo relevantes.
Costo adicional	Aumentar componentes reduce el beneficio de compactar la representación.

El script `analizar_componentes_lsa.R` compara diferentes dimensiones y genera archivos como `comparacion_componentes_lsa.csv`, `ganancia_componentes_lsa.csv` e `informacion_componentes_lsa.csv`. Estos resultados permiten revisar cuánto aporta cada incremento. La decisión final no se basó únicamente en un porcentaje de varianza, sino en el funcionamiento del ranking y en el costo computacional.

10.1 Proyección de la consulta

La consulta no puede compararse directamente con la matriz LSA si permanece en el espacio TF-IDF. Primero se crea su vector TF-IDF y después se aplica la misma transformación utilizada para reducir los documentos. Solo entonces se calcula la similitud coseno.

```
vector_consulta_tfidf <- transformar_consulta_tfidf(
  consulta
)

vector_consulta_lsa <- vector_consulta_tfidf %%% matriz_v

puntajes_lsa <- calcular_similitud_coseno(
  matriz_documentos = matriz_lsa,
  vector_consulta = vector_consulta_lsa
)
```

Decisión metodológica

La reducción sí forma parte del ranking LSA. No se utilizó únicamente para producir una gráfica o una descripción aislada.

11. Estrategia de recuperación TF-IDF

La estrategia TF-IDF es principalmente léxica. Un artículo obtiene un puntaje alto cuando comparte con la consulta términos que además tienen un peso importante dentro del corpus.

Etapas	Operación
Entrada	Consulta escrita por el usuario
Preparación	Normalización, tokenización y stopwords

Representación	Vector TF-IDF con el vocabulario existente
Comparación	Similitud coseno con la matriz TF-IDF
Salida	Artículos ordenados por puntaje

Esta estrategia se espera que funcione especialmente bien cuando la consulta contiene términos presentes de manera directa en títulos y resúmenes. Su principal ventaja es que el resultado puede explicarse observando las palabras compartidas.

12. Estrategia de recuperación LSA

La estrategia LSA parte de la representación TF-IDF, pero proyecta documentos y consultas en 200 componentes. El ranking se calcula en ese espacio reducido.

Etapas	Operación
Entrada	Consulta escrita por el usuario
Preparación	La misma normalización usada en TF-IDF
Representación inicial	Vector TF-IDF
Reducción	Proyección al espacio LSA de 200 componentes
Comparación	Similitud coseno con la matriz LSA
Salida	Artículos ordenados por puntaje LSA

LSA puede relacionar documentos que usan combinaciones distintas de términos, pero su explicación es menos directa porque las componentes no corresponden a palabras individuales. Además, los puntajes de LSA no deben compararse numéricamente con los puntajes de TF-IDF, ya que se calculan en espacios distintos.

13. Comparación de las estrategias

Aspecto	TF-IDF	LSA
Tipo de recuperación	Léxica	Semántica aproximada
Espacio	Vocabulario completo	200 componentes
Matriz	Dispersa	Reducida
Interpretación	Más directa	Menos directa
Fortaleza	Coincidencia de términos	Patrones de coaparición
Riesgo	No reconocer términos relacionados	Perder detalle o incluir relaciones generales
Puntajes	Escala propia TF-IDF	Escala propia LSA

La comparación se realizó mediante rankings y métricas de relevancia. No se compararon los puntajes de forma directa. Una estrategia puede producir puntajes mayores y no ser mejor, porque la escala depende de la representación.



14. Evaluación de resultados

Se definieron cinco consultas relacionadas con el contenido real de la revista. Para cada una se ejecutaron TF-IDF y LSA, se revisaron los cinco primeros artículos y se marcó cada resultado como relevante o no relevante.

Consulta	Tipo
cybersecurity intrusion detection	Consulta específica
generative artificial intelligence and large language models	Términos directos
machine learning for disease diagnosis	Consulta general
social media sentiment analysis	Términos relacionados
statistical methods for prediction	Consulta con resultados poco relevantes

14.1 Precisión

Precisión es la proporción de artículos relevantes dentro de los primeros cinco. Se eligió porque la aplicación muestra un ranking y el interés principal está en la calidad de las primeras posiciones.

Precisión = artículos relevantes en las primeras 5 posiciones / 5

Consulta	Estrategia	Precisión
cybersecurity intrusion detection	LSA	0.80
cybersecurity intrusion detection	TF-IDF	0.80
generative artificial intelligence and large language models	LSA	1.00
generative artificial intelligence and large language models	TF-IDF	1.00
machine learning for disease diagnosis	LSA	1.00
machine learning for disease diagnosis	TF-IDF	1.00
social media sentiment analysis	LSA	1.00
social media sentiment analysis	TF-IDF	1.00
statistical methods for prediction	LSA	0.80
statistical methods for prediction	TF-IDF	0.80

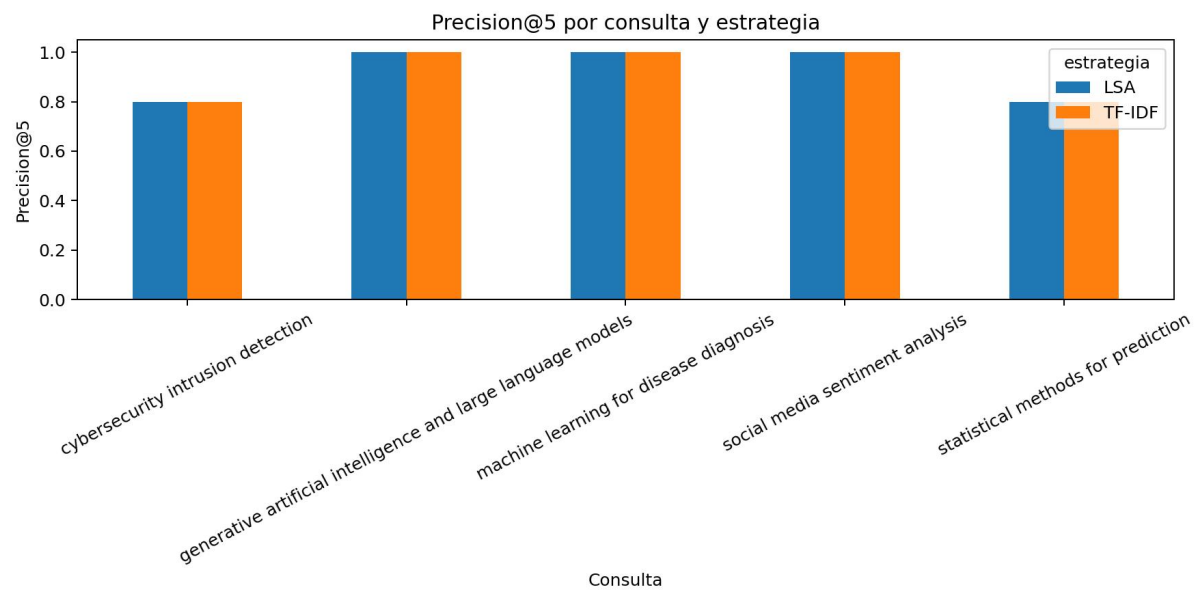


Figura 1. Precisión por consulta y estrategia.

14.2 MRR

MRR evalúa qué tan pronto aparece el primer resultado relevante. Si aparece en la posición 1, el valor es 1; si aparece en la posición 2, el valor es 0,5.

Estrategia	Precisión promedio	MRR
LSA	0.92	0.90
TF-IDF	0.92	0.90

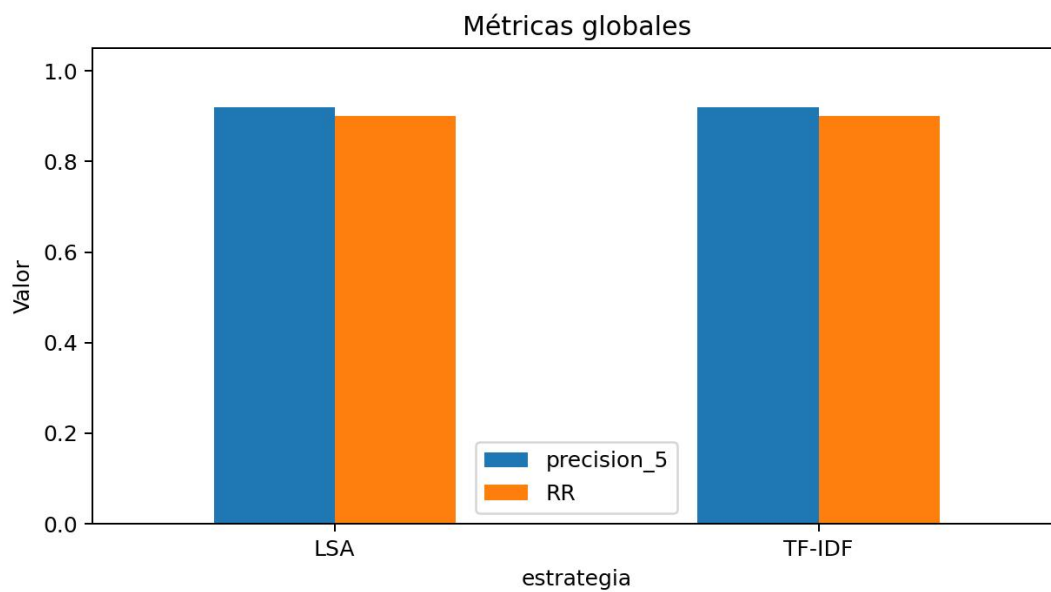


Figura 2. Métricas globales.

Las dos estrategias obtuvieron Precisión promedio de 0,92 y MRR de 0,90. En promedio, 4,6 de los cinco primeros resultados fueron relevantes. El primer resultado útil apareció generalmente en las primeras

posiciones.

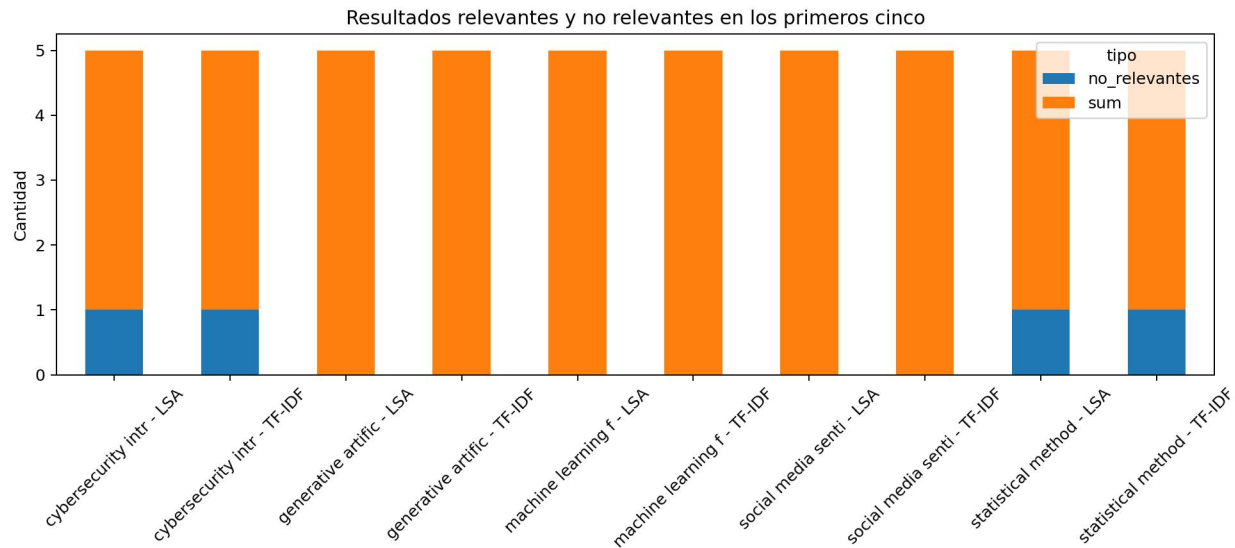


Figura 3. Resultados relevantes y no relevantes.

14.3 Análisis paso a paso de las consultas

cybersecurity intrusion detection

Con LSA, 4 de los cinco resultados fueron considerados relevantes, por lo que Precisión fue 0.80.

Pos.	Artículo	Relevante	Justificación
1	Enhancing the Artificial Rabbit Optimizer Using Fuzzy Rule Interpolation	No	El artículo se centra en optimización mediante lógica difusa y no aborda directamente la detección de intrusiones.
2	A Novel Hybrid IGL1 Feature Selection Method for High-Performance Intrusion Detection on the UNSW-NB15 Dataset Using Multiple Machine Learning Models	Sí	Aborda directamente la detección de intrusiones mediante selección de características y modelos de aprendizaje automático.
3	FedX: Privacy-Preserving Explainable Federated Ensemble Intrusion Detection System for Edge-Enabled Internet of Vehicles	Sí	Presenta un sistema explicable de detección de intrusiones para un entorno de Internet de Vehículos.
4	Neural Network IDS/IPS Intrusion Detection and Prevention System with Adaptive Online Training to Improve Corporate Network Cybersecurity, Evidence Recording, and Interaction with Law Enforcement Agencies	Sí	Se enfoca directamente en la detección y prevención de intrusiones en redes corporativas.
5	A Comparative Study of Federated Learning and Amino Acid Encoding with IoT Malware Detection as a Case Study	Sí	Estudia la detección de malware en IoT, un problema estrechamente relacionado con la ciberseguridad.

Con TF-IDF, 4 de los cinco resultados fueron considerados relevantes, por lo que Precisión fue 0.80.

Pos.	Artículo	Relevante	Justificación
1	A Novel Hybrid IGL1 Feature Selection Method for High-Performance Intrusion Detection on the UNSW-NB15 Dataset Using Multiple Machine Learning Models	Sí	Aborda directamente la detección de intrusiones mediante modelos de aprendizaje automático.

2	FedX: Privacy-Preserving Explainable Federated Ensemble Intrusion Detection System for Edge-Enabled Internet of Vehicles	Sí	Presenta un sistema de detección de intrusiones para entornos vehiculares conectados.
3	Enhancing the Artificial Rabbit Optimizer Using Fuzzy Rule Interpolation	No	El artículo trata optimización y lógica difusa, sin una relación explícita con la detección de intrusiones.
4	Labeling Network Intrusion Detection System (NIDS) Rules with MITRE ATT and CK Techniques: Machine Learning vs. Large Language Models	Sí	Analiza reglas de sistemas de detección de intrusiones con aprendizaje automático y modelos de lenguaje.
5	Neural Network IDS/IPS Intrusion Detection and Prevention System with Adaptive Online Training to Improve Corporate Network Cybersecurity, Evidence Recording, and Interaction with Law Enforcement Agencies	Sí	Se centra directamente en la detección y prevención de intrusiones.

La comparación cualitativa se realizó observando no solo cuántos artículos fueron relevantes, sino también su posición y la razón por la cual el método los recuperó.

generative artificial intelligence and large language models

Con LSA, 5 de los cinco resultados fueron considerados relevantes, por lo que Precisión fue 1.00.

Pos.	Artículo	Relevante	Justificación
1	Generative AI and Large Language Models	Sí	Aborda directamente la inteligencia artificial generativa y los grandes modelos de lenguaje.
2	A Meta-Survey of Generative AI in Education: Trends, Challenges, and Research Directions	Sí	Presenta una revisión sobre tendencias, desafíos y aplicaciones de la inteligencia artificial generativa.
3	The Use of Large Language Models in Ophthalmology: A Scoping Review on Current Use-Cases and Considerations for Future Works in This Field	Sí	Estudia directamente el uso de grandes modelos de lenguaje en el campo de la oftalmología.
4	Generative AI as a General-Purpose Technology: Foundations, Applications, and Labor Market Implications Through 2030	Sí	Analiza fundamentos y aplicaciones de la inteligencia artificial generativa.
5	Exploration of Generative Neural Networks for Police Facial Sketches	Sí	Examina redes neuronales generativas aplicadas a la generación de retratos policiales.

Con TF-IDF, 5 de los cinco resultados fueron considerados relevantes, por lo que Precisión fue 1.00.

Pos.	Artículo	Relevante	Justificación
1	Generative AI and Large Language Models	Sí	Aborda directamente la inteligencia artificial generativa y los grandes modelos de lenguaje.
2	A Meta-Survey of Generative AI in Education: Trends, Challenges, and Research Directions	Sí	Presenta una revisión sobre tendencias, desafíos y aplicaciones de la inteligencia artificial generativa.
3	The Use of Large Language Models in Ophthalmology: A Scoping Review on Current Use-Cases and Considerations for Future Works in This Field	Sí	Estudia directamente el uso de grandes modelos de lenguaje en el campo de la oftalmología.
4	Generative AI as a General-Purpose Technology: Foundations, Applications, and Labor Market Implications Through 2030	Sí	Analiza fundamentos y aplicaciones de la inteligencia artificial generativa.
5	Exploration of Generative Neural Networks for Police Facial Sketches	Sí	Examina redes neuronales generativas aplicadas a la generación de retratos policiales.

La comparación cualitativa se realizó observando no solo cuántos artículos fueron relevantes, sino también su posición y la razón por la cual el método los recuperó.



machine learning for disease diagnosis

Con LSA, 5 de los cinco resultados fueron considerados relevantes, por lo que Precisión fue 1.00.

Pos.	Artículo	Relevante	Justificación
1	Design of an Efficient Model for Psychological Disease Analysis and Prediction Using Machine Learning and Genomic Data Samples	Sí	Utiliza aprendizaje automático para analizar y predecir enfermedades psicológicas.
2	Integrative Machine Learning Framework for Epigenetic Biomarker Discovery and Disease Severity Prediction in Childhood Atopic Dermatitis	Sí	Aplica aprendizaje automático al descubrimiento de biomarcadores y a la predicción de gravedad de una enfermedad.
3	Attention-Driven Deep Learning for News-Based Prediction of Disease Outbreaks	Sí	Utiliza aprendizaje profundo para predecir brotes de enfermedades.
4	Gait-Based Parkinson's Disease Detection Using Recurrent Neural Networks for Wearable Systems	Sí	Aplica redes neuronales para detectar la enfermedad de Parkinson.
5	Efficient Data Augmentation Methods for Crop Disease Recognition in Sustainable Environmental Systems	Sí	Emplea técnicas de aprendizaje automático para reconocer enfermedades en cultivos.

Con TF-IDF, 5 de los cinco resultados fueron considerados relevantes, por lo que Precision fue 1.00.

Pos.	Artículo	Relevante	Justificación
1	Integrative Machine Learning Framework for Epigenetic Biomarker Discovery and Disease Severity Prediction in Childhood Atopic Dermatitis	Sí	Aplica aprendizaje automático al descubrimiento de biomarcadores y a la predicción de gravedad de una enfermedad.
2	Design of an Efficient Model for Psychological Disease Analysis and Prediction Using Machine Learning and Genomic Data Samples	Sí	Utiliza aprendizaje automático para analizar y predecir enfermedades psicológicas.
3	Attention-Driven Deep Learning for News-Based Prediction of Disease Outbreaks	Sí	Utiliza aprendizaje profundo para predecir brotes de enfermedades.
4	Efficient Data Augmentation Methods for Crop Disease Recognition in Sustainable Environmental Systems	Sí	Emplea técnicas de aprendizaje automático para reconocer enfermedades en cultivos.
5	Gait-Based Parkinson's Disease Detection Using Recurrent Neural Networks for Wearable Systems	Sí	Aplica redes neuronales para detectar la enfermedad de Parkinson.

La comparación cualitativa se realizó observando no solo cuántos artículos fueron relevantes, sino también su posición y la razón por la cual el método los recuperó.

social media sentiment analysis

Con LSA, 5 de los cinco resultados fueron considerados relevantes, por lo que Precision fue 1.00.

Pos.	Artículo	Relevante	Justificación
1	EmoBERTa-X: Advanced Emotion Classifier with Multi-Head Attention and DES for Multilabel Emotion Classification	Sí	Trata clasificación de emociones, una tarea estrechamente relacionada con el análisis de sentimiento.
2	An Attention-Based BERT-CNN-BiLSTM Model for Depression Detection from Emojis in Social Media Text	Sí	Analiza depresión a partir de emojis y textos de redes sociales.
3	A Hybrid NER-Sentiment Model for Uzbek Texts: Integrating Lexical, Deep Learning, and Entity-Based Approaches	Sí	Presenta directamente un modelo de análisis de sentimiento.
4	Explicit and Implicit Knowledge in Large-Scale Linguistic Data and Digital Footprints from Social Networks	Sí	Analiza datos lingüísticos y huellas digitales procedentes de redes sociales.
5	SiAraSent: From Features to Deep Transformers for Large-Scale Arabic	Sí	Se centra directamente en análisis de sentimiento

	Sentiment Analysis		a gran escala.
--	--------------------	--	----------------

Con TF-IDF, 5 de los cinco resultados fueron considerados relevantes, por lo que Precision fue 1.00.

Pos.	Artículo	Relevante	Justificación
1	EmoBERTa-X: Advanced Emotion Classifier with Multi-Head Attention and DES for Multilabel Emotion Classification	Sí	Aborda clasificación de emociones, estrechamente relacionada con el análisis de sentimiento.
2	A Hybrid NER–Sentiment Model for Uzbek Texts: Integrating Lexical, Deep Learning, and Entity-Based Approaches	Sí	Presenta directamente un modelo de análisis de sentimiento.
3	Large Language Model-Based Topic-Level Sentiment Analysis for E-Grocery Consumer Reviews	Sí	Realiza análisis de sentimiento sobre opiniones de consumidores.
4	Explainable Deep Learning for COVID-19 Vaccine Sentiment in Arabic Tweets Using Multi-Self-Attention BiLSTM with XLNet	Sí	Analiza sentimiento en publicaciones de Twitter sobre vacunas.
5	An Attention-Based BERT–CNN–BiLSTM Model for Depression Detection from Emojis in Social Media Text	Sí	Estudia textos y emojis de redes sociales mediante modelos de clasificación.

La comparación cualitativa se realizó observando no solo cuántos artículos fueron relevantes, sino también su posición y la razón por la cual el método los recuperó.

statistical methods for prediction

Con LSA, 4 de los cinco resultados fueron considerados relevantes, por lo que Precision fue 0.80.

Pos.	Artículo	Relevante	Justificación
1	Research on Modeling Method of eLoran Signal Propagation Delay Prediction Model: Integrating Path-Weighted Meteorological Data and Propagation Delay Data in Long-Distance Scenarios	Sí	Desarrolla un modelo de predicción de retrasos a partir de datos meteorológicos y de propagación.
2	An Intelligent Evaluation Method for Slope Stability Based on a Database Integrating Real Cases and Numerical Simulations	Sí	Propone un método de evaluación predictiva basado en datos reales y simulaciones.
3	Adversarial Perturbations for Defeating Cryptographic Algorithm Identification	No	Se enfoca en perturbaciones adversariales para identificar algoritmos criptográficos, no en métodos estadísticos de predicción.
4	Attention-Driven Deep Learning for News-Based Prediction of Disease Outbreaks	Sí	Aplica un modelo predictivo para anticipar brotes de enfermedades.
5	Crime Spatiotemporal Prediction Through Urban Region Representation by Using Building Footprints	Sí	Trata directamente la predicción espaciotemporal del crimen.

Con TF-IDF, 4 de los cinco resultados fueron considerados relevantes, por lo que Precision fue 0.80.

Pos.	Artículo	Relevante	Justificación
1	Adversarial Perturbations for Defeating Cryptographic Algorithm Identification	No	No aborda de forma directa métodos estadísticos para predicción.
2	An Intelligent Evaluation Method for Slope Stability Based on a Database Integrating Real Cases and Numerical Simulations	Sí	Desarrolla un método de evaluación predictiva a partir de datos y simulaciones.
3	High-Speed Scientific Computing Using Adaptive Spline Interpolation	Sí	Utiliza interpolación adaptativa, una técnica matemática y estadística de aproximación.

4	Pyramidal Predictive Network V2: An Improved Predictive Architecture and Training Strategies for Future Perception Prediction	Sí	Presenta una arquitectura destinada a la predicción de percepción futura.
5	Adaptive Segmentation and Statistical Analysis for Multivariate Big Data Forecasting	Sí	Aborda explícitamente análisis estadístico y pronóstico de datos multivariados.

La comparación cualitativa se realizó observando no solo cuántos artículos fueron relevantes, sino también su posición y la razón por la cual el método los recuperó.

15. Análisis de resultados incorrectos

Los resultados no relevantes son importantes porque muestran las limitaciones reales. No se eliminaron de la evaluación ni se corrigieron manualmente después de ver las métricas.

cybersecurity intrusion detection - LSA

Artículo: Enhancing the Artificial Rabbit Optimizer Using Fuzzy Rule Interpolation. El artículo se centra en optimización mediante lógica difusa y no aborda directamente la detección de intrusiones.

cybersecurity intrusion detection - TF-IDF

Artículo: Enhancing the Artificial Rabbit Optimizer Using Fuzzy Rule Interpolation. El artículo trata optimización y lógica difusa, sin una relación explícita con la detección de intrusiones.

statistical methods for prediction - LSA

Artículo: Adversarial Perturbations for Defeating Cryptographic Algorithm Identification. Se enfoca en perturbaciones adversariales para identificar algoritmos criptográficos, no en métodos estadísticos de predicción.

statistical methods for prediction - TF-IDF

Artículo: Adversarial Perturbations for Defeating Cryptographic Algorithm Identification. No aborda de forma directa métodos estadísticos para predicción.

En estos casos el método encontró palabras o patrones relacionados con la consulta, pero el objetivo principal del artículo era diferente. Esto confirma que similitud textual y relevancia humana no son exactamente lo mismo.

16. Integración y funcionamiento de la aplicación

La aplicación conserva las pestañas del Taller 2: inicio, estado de la base, indicadores, visualizaciones, tabla de artículos, metodología y actualización SQLite. El Taller 4 agrega la pestaña Buscador inteligente.

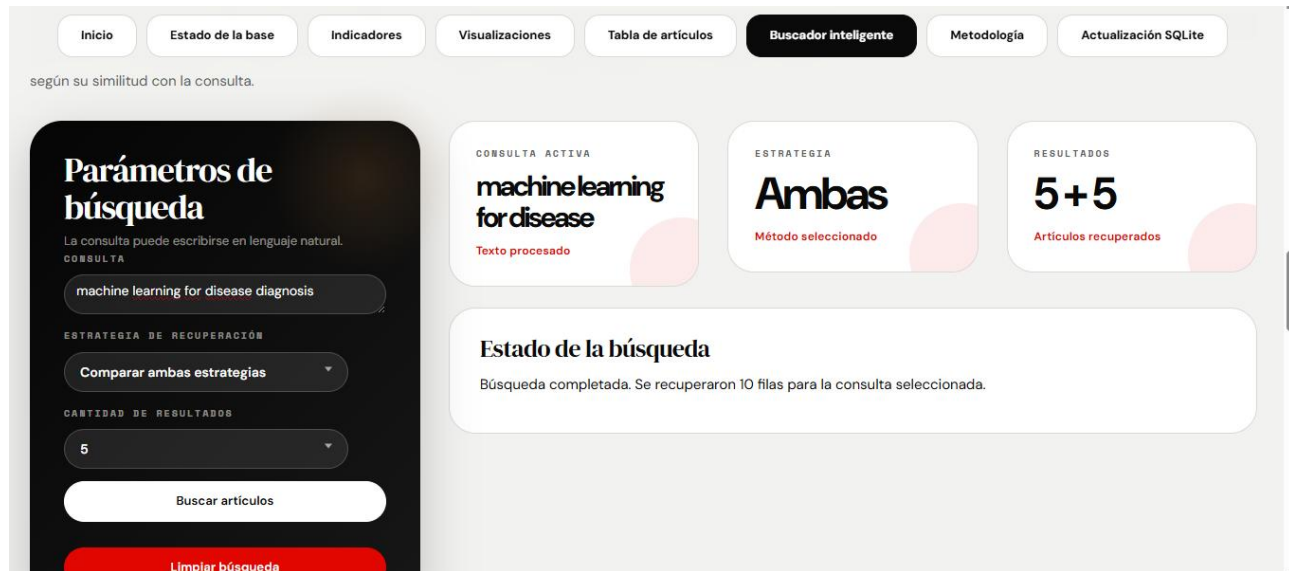


Figura 4. Buscador inteligente desplegado en shinyapps.io.

16.1 Flujo de uso

7. El usuario escribe una consulta en el campo de texto.
8. Selecciona TF-IDF, LSA o la comparación de ambas.
9. Selecciona 5, 10 o 20 resultados.
10. Presiona Buscar artículos.
11. La aplicación ejecuta la función correspondiente.
12. Se actualizan las tarjetas de consulta, estrategia y resultados.
13. La tabla muestra los artículos ordenados.
14. Ver artículo abre el DOI.
15. Ver detalle muestra autores y fragmento.
16. Limpiar búsqueda restablece el módulo.

16.2 Información presentada

Elemento	Función
Consulta activa	Resume el texto que se está procesando
Estrategia	Indica el método seleccionado
Resultados	Muestra la cantidad recuperada
Estado	Confirma si la búsqueda terminó o presenta un error
Tabla	Presenta posición, título, fecha, tema, puntaje, método y enlace
Ver detalle	Muestra autores, fragmento y enlace sin ampliar la tabla
Comparación	Permite observar los rankings de ambos métodos

16.3 Carga de modelos

global.R carga la base SQLite, los objetos TF-IDF, los objetos LSA y las funciones de búsqueda. Esta carga ocurre al iniciar la aplicación. El servidor no reconstruye el corpus ni vuelve a entrenar Truncated SVD por cada consulta.

```
objeto_tfidf <- readRDS(
  "objetos_busqueda/modelo_tfidf_completo.rds"
)

objeto_lsa <- readRDS(
  "objetos_busqueda/modelo_lsa_completo.rds"
)

load(
  "objetos_busqueda/funciones_busqueda_tfidf.RData"
)

load(
  "objetos_busqueda/funciones_busqueda_lsa.RData"
)
```

16.4 Despliegue

La aplicación fue desplegada en shinyapps.io. Para el despliegue se incluyeron ui.R, server.R, global.R, la base SQLite, la carpeta www y los objetos de búsqueda. Los scripts de entrenamiento y los CSV de evaluación se mantienen en GitHub, pero no son necesarios para ejecutar la aplicación publicada.

Enlace: <https://4jqmox-luisa0fernanda-guerrero0ordonez.shinyapps.io/bdcc-research-dashboard-taller4/>

17. Consideraciones computacionales

La solución se diseñó para ejecutarse en un equipo con un máximo de 16 GB de memoria RAM. Para controlar los recursos se utilizaron matrices dispersas, objetos previamente guardados y una representación LSA reducida.



Decisión	Efecto
Matriz TF-IDF dispersa	Evita guardar los ceros como valores densos
Modelos precomputados	Evita reconstruir el corpus en cada consulta
LSA de 200 componentes	Reduce la dimensión para el segundo ranking
RDS y RData	Permiten cargar objetos y funciones de forma rápida
Máximo 20 resultados visibles	Reduce el trabajo de la interfaz
Sin modelos neuronales grandes	Evita dependencias y consumo innecesarios

El análisis de tiempos se realizó repitiendo consultas y registrando el tiempo transcurrido. El análisis de memoria compara `object.size` de la matriz TF-IDF y la matriz LSA. Los archivos `comparacion_tiempos_busqueda.csv` y `resumen_modelo_lsa.csv` conservan esos resultados para el documento reproducible.

18. Reproducibilidad y estructura del proyecto

Archivo	Propósito
<code>global.R</code>	Carga base, modelos y funciones
<code>ui.R</code>	Define la interfaz
<code>server.R</code>	Ejecuta filtros, gráficos y buscador
<code>bdc_2025.sqlite</code>	Base de artículos
<code>construir_corpus.R</code>	Construye y diagnostica el corpus
<code>construir_tfidf.R</code>	Genera vocabulario y matriz TF-IDF
<code>buscar_tfidf.R</code>	Transforma consultas y genera ranking TF-IDF
<code>analizar_componentes_lsa.R</code>	Compara cantidades de componentes
<code>buscar_lsa.R</code>	Proyecta consultas y genera ranking LSA
<code>comparar_estrategias.R</code>	Compara rankings, tiempo y memoria
<code>calcular_precision.R</code>	Calcula Precisión y MRR
<code>plantilla_evaluacion_manual.csv</code>	Registra la valoración humana
<code>objetos_busqueda/</code>	Contiene matrices, corpus, vocabulario, modelos y funciones
<code>www/</code>	Contiene CSS e imagen principal
<code>README.md</code>	Instrucciones y enlace
<code>renv.lock</code>	Dependencias del proyecto

18.1 Orden de reproducción completa

17. Abrir la carpeta taller_4 en RStudio.
18. Restaurar las dependencias.
19. Verificar que bdcc_2025.sqlite esté disponible.
20. Ejecutar preparar_buscadore.R.
21. Ejecutar construir_corpus.R.
22. Ejecutar construir_tfidf.R.
23. Ejecutar buscar_tfidf.R.
24. Ejecutar analizar_componentes_lsa.R.
25. Ejecutar buscar_lsa.R.
26. Ejecutar comparar_estrategias.R.
27. Completar la evaluación manual.
28. Ejecutar calcular_precision.R.
29. Abrir la aplicación con shiny::runApp().

18.2 Ejecución rápida

Cuando los objetos de la carpeta objetos_busqueda ya existen, no es necesario repetir todo el entrenamiento. La aplicación puede iniciarse directamente y cargar esos objetos. Esta es la ruta utilizada en shinyapps.io.

19. Limitaciones

- TF-IDF no reconoce de forma explícita todos los sinónimos.
- LSA puede perder términos específicos durante la reducción.
- La relevancia manual contiene un componente de juicio humano.
- El corpus depende de los campos disponibles en la base.
- Los puntajes de TF-IDF y LSA no son comparables de forma directa.
- Una actualización de la base requiere reconstruir los objetos para que los nuevos artículos formen parte del buscador.
- El almacenamiento local de shinyapps.io no debe considerarse permanente para cambios realizados en SQLite.

20. Conclusiones

El Taller 4 permitió convertir la aplicación del Taller 2 en una herramienta de recuperación de información. El usuario ya no depende únicamente de filtros, sino que puede escribir una consulta temática y recibir artículos ordenados.

TF-IDF fue una elección adecuada como estrategia inicial porque es eficiente, interpretable y funciona bien con términos directos. LSA permitió cumplir el requisito de reducción dimensional y construir un segundo ranking real en un espacio de 200 componentes.

La selección de 200 componentes respondió a un equilibrio entre información, memoria, tiempo y calidad del ranking. La reducción no se presentó como una mejora automática; se evaluó junto con TF-IDF y se reconocieron sus posibles pérdidas.

Las dos estrategias alcanzaron Precisión de 0,92 y MRR de 0,90. El empate global muestra que ambas fueron útiles, pero el análisis por consulta reveló cambios de posición y resultados incorrectos diferentes. Por esa razón, la aplicación permite seleccionar o comparar los métodos.

La integración mediante objetos precomputados fue necesaria para que la aplicación respondiera de forma rápida y pudiera desplegarse. El resultado final conserva el tablero anterior, presenta los campos obligatorios y ofrece una forma clara de revisar autores, fragmentos y DOI.



21. Referencias

Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K. y Harshman, R. (1990). Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6), 391-407.

Manning, C. D., Raghavan, P. y Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

Salton, G. y McGill, M. J. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill.

R Core Team. (2026). *R: A language and environment for statistical computing*.

Posit. Shiny: Web Application Framework for R.

Big Data and Cognitive Computing. Revista utilizada como fuente de los artículos.

Universidad Nacional de Colombia. Enunciado del Taller 4 de Minería de Datos, 2026.

Anexo A. Funcion principal

```
buscar_articulos_tfidf <- function(  
  consulta,  
  n_resultados = 10  
) {  
  
  vector_consulta <- transformar_consulta_tfidf(  
    consulta  
  )  
  
  puntajes <- calcular_similitud_coseno(  
    matriz_tfidf,  
    vector_consulta  
  )  
  
  corpus_articulos %>%  
    mutate(  
      puntaje = puntajes  
    ) %>%  
    arrange(  
      desc(puntaje)  
    ) %>%  
    slice_head(  
      n = n_resultados  
    )  
}
```

Esta función recibe la consulta escrita por el usuario, la transforma a una representación TF-IDF y calcula su similitud con todos los artículos. Después ordena los documentos de mayor a menor puntaje y devuelve la cantidad de resultados seleccionada.

Anexo B. Proyeccion de la consulta en LSA

```
proyectar_consulta_lsa <- function(  
  consulta  
) {  
  
  vector_tfidf <- transformar_consulta_tfidf(  
    consulta  
  )  
  
  vector_tfidf %*%  
    matriz_v  
}
```

Esta función transforma primero la consulta a TF-IDF y luego la proyecta al espacio reducido de LSA. Así, la consulta queda representada en las mismas 200 dimensiones utilizadas para los artículos.

Anexo C. Funcion principal de LSA

```
buscar_articulos_lsa <- function(  
  consulta,  
  n_resultados = 10  
) {  
  
  vector_consulta_lsa <- proyectar_consulta_lsa(  
    consulta  
  )  
}
```

```

norma_consulta <- sqrt(
  sum(
    vector_consulta_lsa ^ 2
  )
)

producto <- as.numeric(
  matriz_lsa %*%
  Matrix::t(
    vector_consulta_lsa
  )
)

puntajes <- producto /
(
  normas_documentos_lsa *
  norma_consulta
)

corpus_articulos %>%
  mutate(
    puntaje = puntajes
  ) %>%
  arrange(
    desc(puntaje)
  ) %>%
  slice_head(
    n = n_resultados
  )
}

```

Esta función calcula la similitud entre la consulta proyectada y la matriz LSA. Luego organiza los artículos según su puntaje y devuelve los primeros resultados del ranking semántico.

Anexo D. Precision

```

precision_5 <- evaluacion %>%
  group_by(
    consulta,
    estrategia
  ) %>%
  summarise(
    relevantes = sum(
      relevante_bin
    ),
    Precision_5 = relevantes / 5,
    .groups = "drop"
  )

```

Este bloque calcula qué proporción de los cinco primeros resultados fue considerada relevante para cada consulta y estrategia. Permite evaluar la calidad inicial del ranking.

Anexo E. MRR

```

mrr <- evaluacion %>%
  filter(
    relevante_bin == 1
  ) %>%
  group_by(
    consulta,
    estrategia
  ) %>%

```

```
summarise(  
  primera_posicion = min(posicion),  
  reciprocal_rank = 1 / primera_posicion,  
  .groups = "drop"  
) %>%  
group_by(  
  estrategia  
) %>%  
summarise(  
  MRR = mean(reciprocal_rank),  
  .groups = "drop"
```

Este bloque identifica la posición del primer resultado relevante en cada consulta. Un valor alto indica que los artículos útiles aparecen desde las primeras posiciones.